



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 29 de marzo
de 2026

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	13
Apéndice B Especificación de Requisitos	15
B.1. Introducción	15
B.2. Objetivos generales	15
B.3. Catálogo de requisitos	16
B.4. Especificación de requisitos	18
Apéndice C Especificación de diseño	27
C.1. Introducción	27
C.2. Diseño de datos	27
C.3. Diseño arquitectónico	43
C.4. Diseño procedimental	43
Apéndice D Documentación técnica de programación	45
D.1. Introducción	45
D.2. Estructura de directorios	45
D.3. Manual del programador	45

D.4. Compilación, instalación y ejecución del proyecto	45
D.5. Pruebas del sistema	45
Apéndice E Documentación de usuario	47
E.1. Introducción	47
E.2. Requisitos de usuarios	47
E.3. Instalación	47
E.4. Manual del usuario	47
Apéndice F Anexo de sostenibilización curricular	49
F.1. Introducción	49
Bibliografía	51

Índice de figuras

A.1. Informe de trabajo completado Sprint 1	3
A.2. Informe de trabajo completado Sprint 2	6
A.3. Informe de trabajo completado Sprint 3	8
A.4. Informe de trabajo completado Sprint 4	9
A.5. Informe de trabajo completado Sprint 5	11
A.6. Informe de trabajo completado Sprint 6	12
C.1. Diagrama entidad-relación del sistema	29
C.2. Diagrama del modelo relacional del sistema	30

Índice de tablas

A.1. Sprint 1 – Resumen	3
A.2. Sprint 2 – Resumen	6
A.3. Sprint 3 – Resumen	8
A.4. Sprint 4 – Resumen	9
A.5. Sprint 5 – Resumen	10
A.6. Sprint 6 – Resumen	12
B.1. CU-MVP-1 Registrarse.	19
B.2. CU-MVP-2 Iniciar sesión.	19
B.3. CU-MVP-3 Cerrar sesión.	20
B.4. CU-MVP-4 Modificar datos usuario.	20
B.5. CU-MVP-5 Ver catálogo permitido de IAs.	21
B.6. CU-MVP-6 Subir imagen y procesar con IA.	22
B.7. CU-MVP-7 Descargar resultado procesado.	23
B.8. CU-MVP-8 Eliminar temporales automáticamente.	24
B.9. CU-MVP-9 Acceder al panel de administración.	25
B.10. CU-MVP-10 Comprobar salud del sistema.	25
C.1. Atributos de la tabla USUARIO	31
C.2. Atributos de la tabla ROL	32
C.3. Atributos de la tabla USUARIO_ROL	33
C.4. Atributos de la tabla TIPO_PLAN	34
C.5. Atributos de la tabla SUSCRIPCION_PLAN	35
C.6. Atributos de la tabla IA_MODELO	36
C.7. Atributos de la tabla IA_MODO	36
C.8. Atributos de la tabla ALQUILA	37
C.9. Atributos de la tabla PIPELINE	38
C.10. Atributos de la tabla EJECUCION	39

<i>Índice de tablas</i>	v
C.11.Atributos de la tabla TEMPORAL_ARCHIVO	40
C.12.Atributos de la tabla PIPELINE_ETAPA	41

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este Trabajo Fin de Grado se ha gestionado con **Scrum**, un marco de trabajo ágil orientado a la entrega iterativa e incremental de valor. Scrum se basa en el *control empírico de procesos* (transparencia, inspección y adaptación) y estructura el desarrollo en iteraciones cortas llamadas *sprints*, con artefactos y eventos bien definidos que favorecen el enfoque, la priorización y la mejora continua.

Scrum

Roles. En este proyecto los roles se han adaptado de forma pragmática:

- **Product Owner (PO):** el propio estudiante, responsable de priorizar el *Product Backlog* alineado con los objetivos del TFG y las indicaciones de los tutores.
- **Scrum Master (SM):** rol asumido también por el estudiante, velando por la correcta aplicación del marco y la eliminación de impedimentos.
- **Equipo de desarrollo:** el estudiante (desarrollo, pruebas y documentación). *Stakeholders:* tutores y tribunal.

Artefactos.

- **Product Backlog:** lista priorizada de épicas e historias (documentación, investigación de modelos de IA de procesamiento de imágenes, pruebas comparativas, etc.).
- **Sprint Backlog:** subconjunto comprometido para cada sprint, desglosado en tareas técnicas.
- **Incremento:** entregables verificables al final de cada sprint (p.ej., secciones de la memoria, scripts de prueba, comparativas).

Eventos.

- **Sprint Planning:** definición del objetivo del sprint y selección de trabajo.
- **Daily Scrum:** chequeo diario (*qué hice/haré/impedimentos*) para mantener foco.
- **Sprint Review:** revisión de avances con evidencias (código, resultados, documentación).
- **Sprint Retrospective:** lecciones aprendidas y acciones de mejora para el siguiente sprint.

Medición y estimación del trabajo

Para la planificación y seguimiento del progreso se ha empleado un sistema de medición basado en puntos de historia, una unidad habitual en metodologías ágiles como Scrum. Cada tarea o ítem del *Sprint Backlog* se ha valorado con un número de puntos que refleja su esfuerzo y complejidad relativa.

En este proyecto, se ha establecido una equivalencia práctica de 8 puntos por jornada completa de trabajo, lo que permite estimar la carga de trabajo de cada sprint y evaluar el avance real frente al planificado. Este sistema facilita mantener una visión objetiva del esfuerzo invertido y ajustar la planificación de los siguientes sprints en función de la velocidad media alcanzada.

A.2. Planificación temporal

La planificación se organiza en sprints cortos (1–2 semanas).

Visión general del Sprint 1

- **Fechas:** 27/10 – 06/11
- **Objetivo del sprint:** *Investigación y pruebas iniciales de IAs de procesado de imágenes.*
- **Criterio de aceptación global:** disponer de dos IAs instaladas y probadas con evidencias, y una comparativa preliminar documentada.

Tal y como se muestra en la Tabla A.1 y en la Figura A.1, este sprint se centra en la fase inicial de investigación y preparación del proyecto.

Tarea	Pts
Organización inicial del proyecto	8
Aprendizaje básico de L ^A T _E X	8
Estructura base del documento TFG	8
Investigar modelos IA de imágenes	8
Instalar y probar 1ª IA	8
Instalar y probar 2ª IA	8
Comparativa entre IAs seleccionadas	8
Cierre Sprint	8

Tabla A.1: Sprint 1 – Resumen



Figura A.1: Informe de trabajo completado Sprint 1

Descripción de las tareas del Sprint 1

A continuación se describen brevemente las tareas que componen el Sprint 1:

- **Organización inicial del proyecto.** Se configuró el entorno de trabajo creando la estructura del repositorio, las carpetas para código, experimentos, documentación y resultados, así como los archivos necesarios para el control de versiones y la vinculación con Jira. Esta tarea permitió establecer la base técnica sobre la que se desarrollará el resto del proyecto.
- **Aprendizaje básico de $\text{\LaTeX}$.** Se dedicó tiempo a adquirir soltura con la herramienta de edición científica $\text{\LaTeX}$, aprendiendo a crear capítulos, incluir figuras, tablas, referencias y bibliografía. Este conocimiento asegura una correcta elaboración y mantenimiento de la memoria del TFG.
- **Estructura base del documento TFG.** Se definió la organización general de la memoria, creando los capítulos principales, anexos y secciones iniciales. De este modo quedó preparada la estructura en la que se irá incorporando el contenido teórico y técnico conforme avance el proyecto.
- **Investigar modelos IA de procesamiento de imágenes.** Se llevó a cabo una investigación de diferentes alternativas de inteligencia artificial aplicadas al procesamiento de imágenes, evaluando su documentación, facilidad de uso, licencia y compatibilidad. Tras este estudio se seleccionaron dos herramientas destacadas: **YOLO** y **MediaPipe**, por su relevancia en el ámbito de la visión por computador.
- **Instalar y probar la primera IA seleccionada (YOLO).** En esta tarea se instaló y configuró el modelo YOLO (You Only Look Once), mediante la librería `ultralytics` en Python. Se realizaron pruebas de detección de objetos sobre imágenes propias para verificar la correcta ejecución del modelo, su velocidad de inferencia y la claridad de sus resultados. Esta fase permitió comprobar la idoneidad de YOLO para el reconocimiento y localización de elementos dentro de una escena.
- **Instalar y probar la segunda IA seleccionada (MediaPipe).** Posteriormente se instaló el framework MediaPipe de Google, que ofrece soluciones preentrenadas orientadas al análisis de características humanas como el rostro, las manos o la pose corporal. Se probaron

distintos módulos, especialmente los de detección de manos y pose, verificando su funcionamiento en tiempo real y su facilidad de integración en proyectos Python.

- **Comparativa entre las IAs seleccionadas.** Una vez probadas ambas herramientas, se elaboró una comparativa que analiza sus principales diferencias: finalidad, precisión, velocidad, facilidad de uso, requerimientos y tipo de tareas para las que son más adecuadas. En esta comparativa se concluye que **MediaPipe** destaca en la detección y seguimiento de características humanas con bajo coste computacional, mientras que **YOLO** ofrece mayor versatilidad para la detección general de objetos y escenarios más complejos. La documentación y análisis detallados de esta comparativa se incluyen en el capítulo 4 de la memoria.
- **Cierre del Sprint.** Finalmente, se revisaron todas las tareas completadas y se documentaron los avances obtenidos, preparando el tablero y el backlog para el siguiente sprint. Esta revisión permitió valorar el cumplimiento de los objetivos y establecer las líneas de trabajo futuras.

Visión general del Sprint 2

- **Fechas:** 09/11 – 20/11
- **Objetivo del sprint:** *Construir un servidor funcional en Flask capaz de recibir imágenes, ejecutar detecciones simuladas y ofrecer una interfaz mínima de usuario.*
- **Criterio de aceptación global:** disponer de un servidor operativo, con endpoints funcionales, selección de modelos simulada y una UI básica que permita validar el flujo completo de subida y procesado.

Tal y como se recoge en la Tabla A.2 y en la Figura A.2, este sprint introduce la construcción del servidor y la primera interfaz funcional.

Descripción de las tareas del Sprint 2

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Definir alcance del sprint y crear historias.** Se concretaron los objetivos del sprint y se desglosaron en historias de usuario.

Tarea	Pts
Definir alcance del sprint y crear historias	8
Elegir framework servidor Flask	8
Elección librería para simulación de detecciones	6
Configurar entorno desarrollo y estructura base	6
Probar funcionamiento servidor	8
Desarrollar endpoint en servidor para subir imágenes	10
Desarrollar endpoint en servidor para detección simulada	12
Crear interfaz mínima de aplicación	8
Validación final y documentación del sprint	6

Tabla A.2: Sprint 2 – Resumen

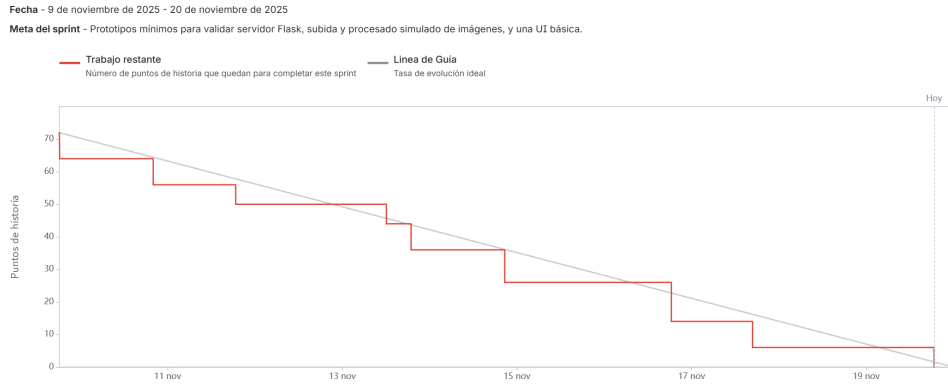


Figura A.2: Informe de trabajo completado Sprint 2

- **Elegir framework servidor.** Tras evaluar distintas opciones, se seleccionó **Flask** como framework principal por su ligereza, modularidad y facilidad para integrar endpoints REST orientados al procesado de imágenes.
- **Elección librería para simulación de detecciones.** Para poder validar el flujo completo antes de integrar YOLO o MediaPipe reales, se creó una librería propia que genera detecciones simuladas (clases, cajas y confianza).
- **Configurar entorno de desarrollo y estructura base.** Se construyó una arquitectura inicial organizada en módulos y se configuró un servidor Flask estable listo para ampliaciones.

- **Probar funcionamiento del servidor.** Se verificó la ejecución del backend, comprobando rutas, carga de configuraciones, manejo de errores y logs. Esta fase permitió asegurar que el servidor respondía correctamente antes de construir los endpoints principales.
- **Desarrollar endpoint para subir imágenes.** Se creó un endpoint capaz de recibir imágenes vía POST, almacenarlas temporalmente y devolver su ruta accesible desde la interfaz.
- **Desarrollar endpoint para detección.** Se implementó un endpoint `/detect` que recibe la imagen subida y devuelve detecciones generadas mediante la librería simulada y posteriormente los endpoint para el procesamiento de imágenes con yolo y mediapipe aplicando patrón factory.
- **Crear interfaz mínima de aplicación.** Se desarrolló una pequeña UI con HTML y JavaScript para permitir al usuario subir imágenes, elegir un modelo disponible y visualizar el resultado generado.
- **Validación final y documentación del sprint.** Se revisaron todas las historias completadas, se generaron capturas de evidencia y se documentó el sprint.

Visión general del Sprint 3

- **Fechas:** 14/02 – 19/02
- **Objetivo del sprint:** *Definir un borrador de los requisitos funcionales, requisitos no funcionales y casos de uso del proyecto.*
- **Criterio de aceptación global:** tener suficientes requisitos y casos de uso para cubrir el funcionamiento total de la aplicación.

Como se observa en la Tabla A.3 y la Figura A.3, este sprint se centra en la definición de requisitos y casos de uso.

Descripción de las tareas del Sprint 3

Nada importante a destacar en este sprint.

Tarea	Pts
Definición alcance sprint	2
Plantear alcance de requisitos y casos de uso	6
Definir requisitos funcionales	8
Definir requisitos no funcionales	8
Definir casos de uso	16

Tabla A.3: Sprint 3 – Resumen

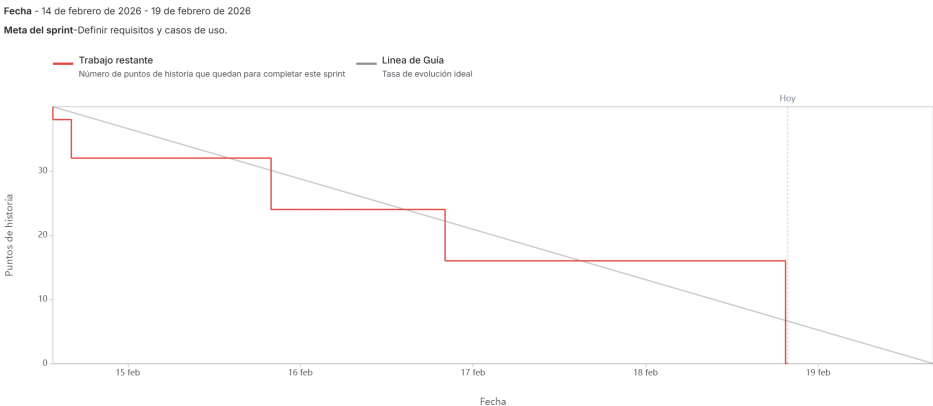


Figura A.3: Informe de trabajo completado Sprint 3

Visión general del Sprint 4

- **Fechas:** 19/02 – 25/02
- **Objetivo del sprint:** *Completar la documentación del MVP (RF/RNF/CU) y diseñar el modelo E-R de la aplicación (entidades, atributos, relaciones, normalización y reglas).*
- **Criterio de aceptación global:** disponer de la especificación de requisitos del MVP documentada y un modelo E-R completo y coherente (incluyendo normalización, integridad y restricciones de negocio).

Tal y como se muestra en la Tabla A.4 y la Figura A.4, este sprint se enfoca en el diseño del modelo de datos y la documentación del MVP.

Tarea	Pts
Corregir y subir documento “RF_RNF_CU_aplicación_completa”	2
Documentar los RF/RNF/CU para el producto mínimo viable	6
Modelo E-R (identificación de entidades y alcance)	8
Modelo E-R (atributos y claves)	8
Modelo E-R (relaciones y cardinalidades)	8
Modelo E-R (normalización, reglas de integridad y restricciones de negocio)	8

Tabla A.4: Sprint 4 – Resumen

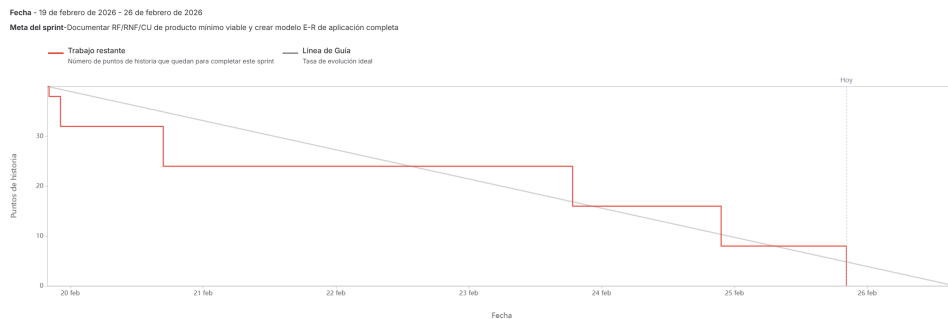


Figura A.4: Informe de trabajo completado Sprint 4

Descripción de las tareas del Sprint 4

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Corregir y subir documento “RF_RNF_CU_aplicación_completa”.**
Se revisó el documento general de requisitos y casos de uso de la aplicación completa, corrigiendo inconsistencias de redacción y estructura.
- **Documentar los RF/RNF/CU para el producto mínimo viable.**
Se seleccionaron y redactaron los requisitos funcionales y no funcionales junto con los casos de uso estrictamente necesarios para el MVP, priorizando el flujo básico de acceso, selección de IA, envío de imagen y obtención de resultados.
- **Modelo E-R (identificación de entidades y alcance).** Se identificaron las entidades completas de la aplicación y se delimitó el alcance del modelo para cubrir tanto el MVP como la aplicación completa.

- **Modelo E-R (atributos y claves).** Se definieron los atributos relevantes de cada entidad y sus claves, estableciendo claves primarias, restricciones de unicidad y una estructura consistente para su posterior representación en el diagrama.
- **Modelo E-R (relaciones y cardinalidades).** Se diseñó el diagrama E-R utilizando las entidades con sus atributos creados previamente junto definiendo relaciones y cardinalidades que unirían estas.
- **Modelo E-R (normalización, reglas de integridad y restricciones de negocio).** Se verificó el modelo respecto a formas normales (hasta 3FN) y se definieron reglas de integridad (entidad, referencial y coherencia semántica), además de restricciones de negocio (dominios de estado, coherencia temporal, unicidades y reglas específicas como renovaciones y consistencia IA–modo).

Visión general del Sprint 5

- **Fechas:** 03/03 – 05/03
- **Objetivo del sprint:** *Retroalimentación de diagramas E-R y Relacional.*
- **Criterio de aceptación global:** Tener correcciones hechas sobre los modelos E-R y Relacional de acuerdo a lo comentado en el Sprint Review.

Como se refleja en la Tabla A.5 y la Figura A.5, este sprint se centra en la mejora de los modelos E-R y relacional.

Tarea	Pts
Modificar modelo relacional	8
Modelo E-R	8
Repasar si E-R y relacional son válidos para casos de uso	8

Tabla A.5: Sprint 5 – Resumen

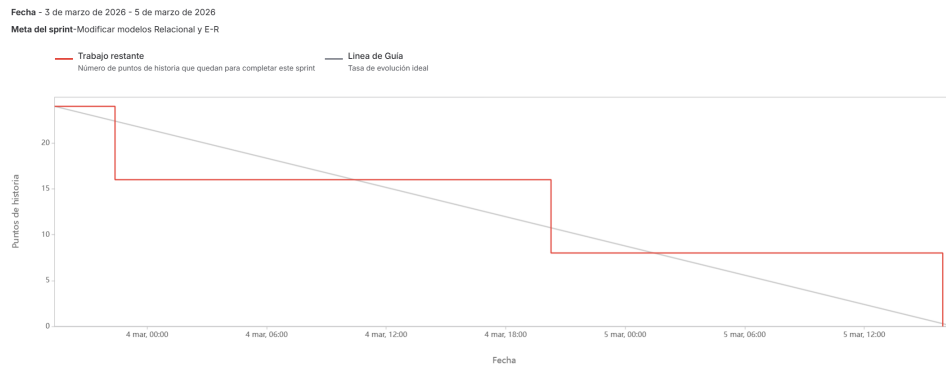


Figura A.5: Informe de trabajo completado Sprint 5

Descripción de las tareas del Sprint 5

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Modificar modelo relacional.** Hacer modificaciones en el modelo relacional para poder estar totalmente normalizado y tener una mejor estructura en cuanto a crecimiento de aplicación.
- **Modelo E-R.** Realizar el modelo Entidad Relación con la retrospectiva del Sprint Review.
- **Repasar si E-R y relacional son válidos para casos de uso.** Verificar si con ambos diagramas se podrían llegar a realizar correctamente todos los casos de usos planteados en anteriores sprints.

Visión general del Sprint 6

- **Fechas:** 07/03 – 12/03
- **Objetivo del sprint:** *Definir y ajustar el modelo de configuración y ejecución de pipelines.*
- **Criterio de aceptación global:** Tener definidos los criterios de configuración, creación de pipelines y ejecuciones, y reflejarlo en el modelo de datos.

Tal y como se muestra en la Tabla A.6 y la Figura A.6, este sprint introduce el modelo de pipelines y su configuración.

Tarea	Pts
Modificación E-R	4
Plantear si habrá configuración elegible por el usuario o no	6
Plantear si los pipelines los crea el administrador o usuarios	2
Plantear si se pueden repetir ejecuciones	2
Actualizar modelo respecto a los planteamientos tomados	10
Revisar si los requisitos y casos de uso cuadran con respecto a los planteamientos	8

Tabla A.6: Sprint 6 – Resumen

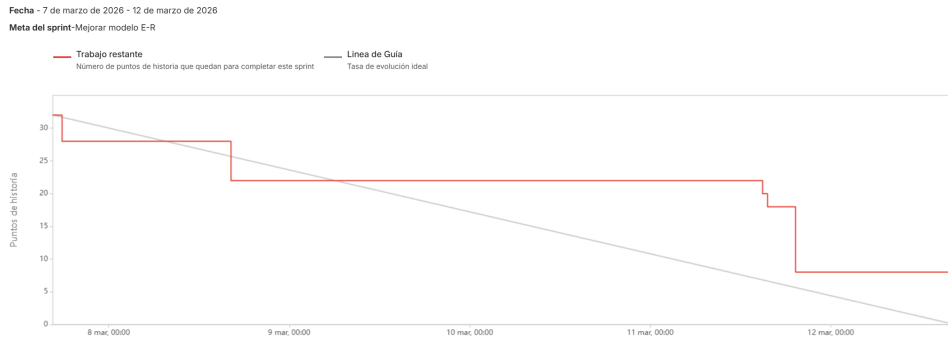


Figura A.6: Informe de trabajo completado Sprint 6

Descripción de las tareas del Sprint 6

A continuación se describen brevemente las tareas realizadas durante este sprint:

- **Modificación E-R.** Ajustar el diagrama E-R según los nuevos planteamientos.
- **Plantear si habrá configuración elegible por el usuario o no.** Analizar si los usuarios pueden modificar configuraciones.
- **Plantear si los pipelines los crea el administrador o usuarios.** Determinar si los pipelines los crea el administrador o los usuarios.
- **Plantear si se pueden repetir ejecuciones.** Estudiar si las ejecuciones pueden repetirse.
- **Actualizar modelo respecto a los planteamientos tomados.** Reflejar las decisiones tomadas en el modelo de datos.

- **Revisar si los requisitos y casos de uso cuadran con respecto a los planteamientos.** Comprobar coherencia entre requisitos, casos de uso y modelo.

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge los requisitos y casos de uso del **Producto Mínimo Viable (MVP)** del sistema. El MVP se centra en: *autenticación de usuarios (Básico, Pro y Administrador)*, *selección de IA*, *subida de imagen*, *procesamiento y descarga del resultado*, con despliegue funcional para demostración. Se excluyen funcionalidades no imprescindibles (por ejemplo, historial de compras, pipelines de IAs, auditorías avanzadas).

B.2. Objetivos generales

- Permitir el acceso de usuarios autenticados (Básico, Pro y Administrador) a una plataforma web.
- Ofrecer un flujo completo: **seleccionar Pipeline de IAs y modos** → **subir imagen** → **procesar** → **visualizar resultado** → **descargar**.
- Aplicar control de acceso en servidor según el plan (Básico/Pro) y rol (Administrador).
- Garantizar que las imágenes y resultados sean **temporales**.
- Disponer de un despliegue reproducible para que la aplicación esté operativa (demo).

B.3. Catálogo de requisitos

Requisitos funcionales (MVP)

RF-MVP-1 Gestión de cuentas de usuario

La aplicación debe permitir registrar, iniciar sesión y cerrar sesión para acceder al sistema.

- RF-MVP-1.1 Registro de usuario.
- RF-MVP-1.2 Inicio de sesión.
- RF-MVP-1.3 Cierre de sesión.
- RF-MVP-1.4 Modificar datos de cuenta.

RF-MVP-2 Roles y planes (Básico, Pro, Administrador)

La aplicación debe distinguir los tres tipos de usuario y aplicar permisos en servidor.

- RF-MVP-2.1 Rol Administrador con acceso a panel de gestión.
- RF-MVP-2.2 Plan Básico: acceso únicamente a IAs activas para ese usuario.
- RF-MVP-2.3 Plan Pro: acceso a todas las IAs disponibles.
- RF-MVP-2.4 Visualización del plan actual (Básico/Pro) y estado.

RF-MVP-3 Catálogo de IAs y modos (según permisos)

La aplicación debe mostrar y permitir seleccionar únicamente pipelines con IAs y modos permitidos.

- RF-MVP-3.1 Catálogo general de pipelines de IAs disponibles en el sistema.
- RF-MVP-3.2 Modos asociados a cada IA.
- RF-MVP-3.3 Filtrado del catálogo por plan/rol en la interfaz.

RF-MVP-4 Subida puntual de imagen

La aplicación debe permitir subir una imagen para procesarla sin persistencia.

- RF-MVP-4.1 Subir imagen desde el panel de análisis.
- RF-MVP-4.2 Validación básica (tipo y tamaño máximo).
- RF-MVP-4.3 Uso temporal y eliminación automática tras el procesamiento.

RF-MVP-5 Procesamiento con IA, visualización y descarga

La aplicación debe ejecutar el análisis con la IA seleccionada y permitir descargar el resultado.

- RF-MVP-5.1 Selección de pipeline (IAs y modos).
- RF-MVP-5.2 Ejecución del procesamiento.
- RF-MVP-5.3 Mostrar imagen procesada como resultado principal.
- RF-MVP-5.4 Mostrar JSON del análisis como resultado secundario.
- RF-MVP-5.5 Descargar la imagen procesada.

RF-MVP-6 Restricciones y manejo de errores

La aplicación debe impedir acciones no permitidas y mostrar mensajes claros.

- RF-MVP-6.1 Impedir ejecutar IAs no permitidas.
- RF-MVP-6.2 Impedir procesar si no hay sesión iniciada.
- RF-MVP-6.3 Error de imagen inválida: informar y no procesar.
- RF-MVP-6.4 Error interno: informar sin detalles técnicos.

RF-MVP-7 Interfaz web del flujo principal

La aplicación debe disponer de una interfaz web centrada en el flujo del MVP.

- RF-MVP-7.1 Pantallas: registro, login, ajustes, panel de análisis, resultado y descarga.
- RF-MVP-7.2 Selectores dinámicos: modos cambian según IA.

RF-MVP-8 API mínima (soporte de demo e integración)

La aplicación debe exponer endpoints básicos para comprobación y ejecución.

- RF-MVP-8.1 Endpoint de salud del sistema.
- RF-MVP-8.2 Endpoint de catálogo permitido según usuario.
- RF-MVP-8.3 Endpoint de ejecución de análisis (imagen, IA, modo).

RF-MVP-9 Administración mínima

El administrador debe poder ver usuarios y su plan para soporte básico.

- RF-MVP-9.1 Listado de usuarios con plan/rol.
- RF-MVP-9.2 Ver detalle de usuario (plan/estado).

Requisitos no funcionales (MVP)

RNF-MVP-1 Usabilidad

Interfaz clara, centrada en el flujo principal, con mensajes comprensibles.

RNF-MVP-2 Rendimiento

Procesamiento en tiempos razonables y limitación de tamaño de imagen.

RNF-MVP-3 Seguridad

Autenticación obligatoria, aislamiento entre usuarios y validación de ficheros.

RNF-MVP-4 Portabilidad y despliegue

Ejecución reproducible (local o servidor) con configuración centralizada y dependencias documentadas.

B.4. Especificación de requisitos

CU-MVP-1	Registrarse
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-1.1
Descripción	Creación de una cuenta para acceder a la aplicación.
Precondición	El usuario no dispone de cuenta.
Acciones	1. El usuario accede a la pantalla de registro. 2. Introduce los datos requeridos. 3. El sistema valida los datos. 4. El sistema crea la cuenta. 5. El sistema informa del éxito y redirige a inicio de sesión.
Postcondición	Cuenta creada.
Excepciones	Email o usuario ya existe; datos inválidos.
Importancia	Alta

Tabla B.1: CU-MVP-1 Registrarse.

CU-MVP-2	Iniciar sesión
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-1.2, RF-MVP-6.2
Descripción	Acceso del usuario a la aplicación con sus credenciales.
Precondición	Cuenta creada.
Acciones	1. El usuario accede a la pantalla de inicio de sesión. 2. Introduce credenciales. 3. El sistema valida. 4. El sistema inicia sesión y redirige al panel de análisis.
Postcondición	Sesión iniciada.
Excepciones	Credenciales incorrectas; cuenta desactivada.
Importancia	Alta

Tabla B.2: CU-MVP-2 Iniciar sesión.

CU-MVP-3	Cerrar sesión
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-1.3
Descripción	Salida del usuario del sistema.
Precondición	Sesión iniciada.
Acciones	1. El usuario pulsa cerrar sesión. 2. El sistema finaliza la sesión. 3. Redirige a la pantalla de inicio de sesión.
Postcondición	Sesión cerrada.
Excepciones	No aplica.
Importancia	Media

Tabla B.3: CU-MVP-3 Cerrar sesión.

CU-MVP-4	Modificar datos usuario.
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-1.4
Descripción	El usuario debe poder visualizar y cambiar sus datos.
Precondición	Sesión iniciada.
Acciones	1. El usuario pulsa ajustes. 2. El sistema redirige a la página de ajustes de dicho usuario. 3. El usuario ve sus datos. 4. El usuario selecciona campo a cambiar. 5. El sistema realiza el cambio. 6. El usuario ve el cambio realizado.
Postcondición	Datos modificados.
Excepciones	Cambiar contraseña pide la actual.
Importancia	Baja

Tabla B.4: CU-MVP-4 Modificar datos usuario.

CU-MVP-5	Ver catálogo permitido de IAs
Versión	1.0
Actor	Sistema
Requisitos asociados	RF-MVP-2.2, RF-MVP-2.3, RF-MVP-3.1, RF-MVP-3.2, RF-MVP-3.3
Descripción	Muestra al usuario sólo los pipelines de las IAs y modos a los que tiene acceso según su plan.
Precondición	Sesión iniciada.
Acciones	1. El usuario accede al panel de análisis. 2. El sistema obtiene el catálogo permitido para el usuario. 3. La interfaz muestra únicamente las IAs disponibles.
Postcondición	Catálogo visible y listo para usar.
Excepciones	En Básico sin IAs activas: se muestra aviso de no disponibilidad.
Importancia	Muy alta

Tabla B.5: CU-MVP-5 Ver catálogo permitido de IAs.

CU-MVP-6	Subir imagen y procesar con IA
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-4.1, RF-MVP-4.2, RF-MVP-4.3, RF-MVP-5.1, RF-MVP-5.2, RF-MVP-5.3, RF-MVP-5.4, RF-MVP-6.1, RF-MVP-6.3, RF-MVP-6.4
Descripción	Flujo principal: subir imagen, elegir IA y modo, procesar y ver resultado sin historial.
Precondición	Sesión iniciada y acceso vigente a la IA seleccionada.
Acciones	1. El usuario accede al panel de análisis. 2. Adjunta una imagen del dispositivo. 3. Selecciona pipeline con IAs y modos disponibles. 4. Pulsa <i>Analizar</i>. 5. El sistema valida tipo y tamaño de la imagen. 6. El sistema valida permisos (plan/rol) para la IA seleccionada. 7. El sistema ejecuta el análisis. 8. El sistema muestra la imagen procesada como resultado principal. 9. El sistema muestra el JSON del análisis como resultado secundario.
Postcondición	Resultado mostrado en pantalla (disponible temporalmente).
Excepciones	Imagen inválida; IA no permitida; error interno.
Importancia	Muy alta

Tabla B.6: CU-MVP-6 Subir imagen y procesar con IA.

CU-MVP-7	Descargar resultado procesado
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-5.5
Descripción	Permite descargar la imagen procesada obtenida en la ejecución actual.
Precondición	Haber completado un procesamiento con resultado disponible.
Acciones	1. El usuario pulsa <i>Descargar</i>. 2. El sistema envía el archivo procesado para descarga.
Postcondición	Archivo descargado.
Excepciones	Resultado no encontrado o caducado: se informa y se propone repetir el análisis.
Importancia	Alta

Tabla B.7: CU-MVP-7 Descargar resultado procesado.

CU-MVP-8	Eliminar temporales automáticamente
Versión	1.0
Actor	Sistema
Requisitos asociados	RF-MVP-4.3
Descripción	El sistema elimina automáticamente ficheros temporales de subida y resultado para no crear historial.
Precondición	Haber existido una ejecución o haber pasado el tiempo máximo configurado.
Acciones	1. El sistema marca los ficheros temporales como eliminables. 2. Tras finalizar la ejecución o superar el tiempo máximo, el sistema elimina los ficheros temporales. 3. El sistema evita acumulación de archivos.
Postcondición	No quedan imágenes/resultados persistidos como historial.
Excepciones	Si falla la eliminación: se registra el error para mantenimiento.
Importancia	Alta

Tabla B.8: CU-MVP-8 Eliminar temporales automáticamente.

CU-MVP-9	Acceder al panel de administración
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-2.1, RF-MVP-9.1, RF-MVP-9.2
Descripción	Acceso del administrador a funciones mínimas de soporte.
Precondición	Sesión iniciada como Administrador.
Acciones	1. El administrador accede al panel de administración. 2. El sistema muestra el listado de usuarios. 3. El administrador selecciona un usuario para ver su detalle.
Postcondición	Panel de administración operativo.
Excepciones	Usuario no administrador: acceso denegado.
Importancia	Alta

Tabla B.9: CU-MVP-9 Acceder al panel de administración.

CU-MVP-10	Comprobar salud del sistema
Versión	1.0
Actor	Usuario
Requisitos asociados	RF-MVP-8.1
Descripción	Permite comprobar que el servidor está operativo mediante un endpoint de salud.
Precondición	Servidor en ejecución.
Acciones	1. Un usuario técnico o herramienta consulta el endpoint de salud. 2. El sistema responde correctamente.
Postcondición	Estado verificado.
Excepciones	Si no responde: se asume caída.
Importancia	Media Alta

Tabla B.10: CU-MVP-10 Comprobar salud del sistema.

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se describe el diseño técnico del sistema, estructurado en tres apartados: diseño de datos, diseño arquitectónico y diseño procedimental.

El objetivo de este anexo es justificar las decisiones adoptadas durante el diseño de la solución. En particular, en el diseño de datos no solo se presenta la estructura final de la base de datos, sino también el motivo por el que se han definido determinados atributos, restricciones y relaciones. De este modo, el modelo no se plantea como una simple colección de tablas, sino como una propuesta coherente con los requisitos del sistema, la trazabilidad de la información y la posibilidad de ampliación futura.

C.2. Diseño de datos

El modelo de datos se ha diseñado para su implantación en MariaDB utilizando el motor InnoDB, lo que permite trabajar con claves foráneas, integridad referencial y soporte transaccional. Además, se utiliza `utf8mb4` con colación `utf8mb4_unicode_ci` para permitir el almacenamiento correcto de caracteres Unicode.

De manera general, el diseño se ha construido siguiendo varios criterios:

- **Separación de responsabilidades:** cada tabla representa una entidad o una asociación con significado propio.

- **Evitar redundancia innecesaria:** se distinguen claramente conceptos como usuarios, roles, planes, suscripciones, modelos IA, modos, pipelines y ejecuciones.
- **Preservar la integridad:** se emplean claves primarias, claves foráneas, restricciones `UNIQUE` y restricciones `CHECK`.
- **Conservar la trazabilidad:** se evita el borrado automático en cascada en aquellas tablas cuyo contenido forma parte del histórico del sistema.
- **Facilitar la escalabilidad:** la estructura permite añadir nuevos planes, nuevos modelos IA, nuevos modos o nuevos pipelines sin modificar la base del modelo.

Antes de justificar cada una de las tablas del modelo, se incluyen dos representaciones complementarias del diseño de datos. En primer lugar, se presenta el diagrama entidad-relación, que permite comprender la estructura conceptual del sistema, sus entidades principales y las asociaciones existentes entre ellas. En segundo lugar, se muestra el diagrama del modelo relacional, donde dicha propuesta conceptual queda traducida a tablas, claves y relaciones concretas para su implantación en la base de datos.

Diagrama entidad-relación

La Figura C.1 recoge el diagrama entidad-relación del sistema. En él se representa la visión conceptual del modelo de datos, identificando las entidades principales, sus atributos más relevantes y las relaciones que se establecen entre ellas.

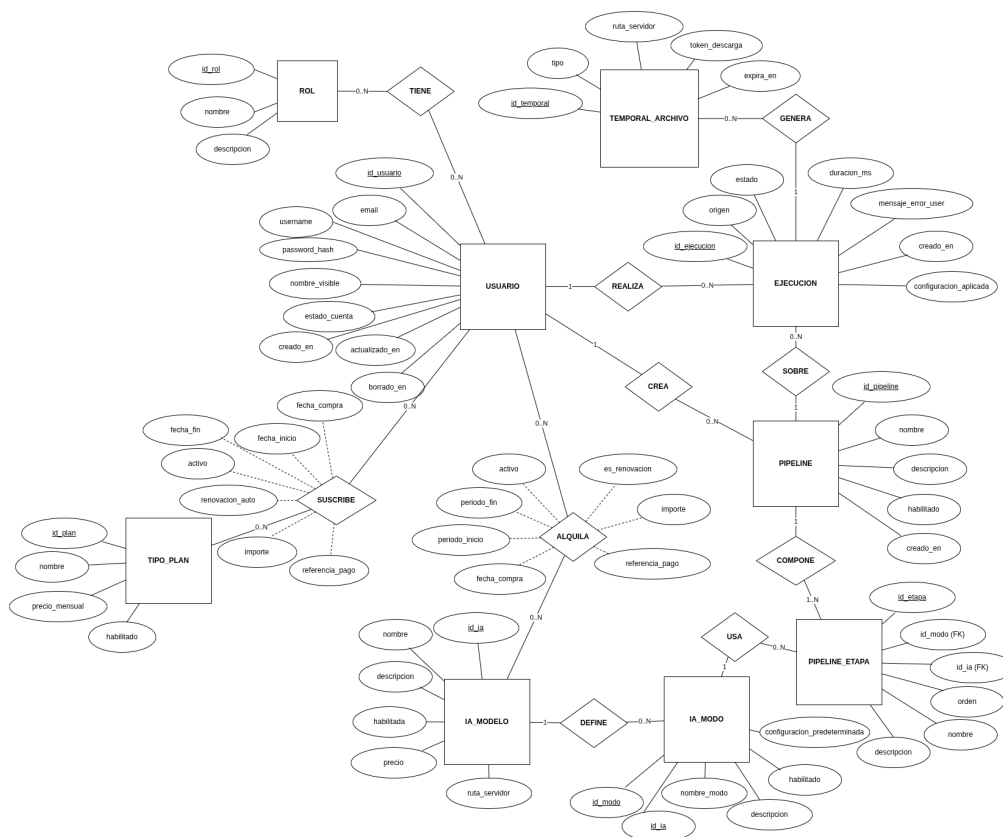


Figura C.1: Diagrama entidad-relación del sistema

Este diagrama permite observar que **USUARIO** actúa como entidad central del sistema, ya que se relaciona con la asignación de roles, la contratación de planes, el acceso individual a modelos de inteligencia artificial, la creación de pipelines y la realización de ejecuciones. Asimismo, se distingue la parte funcional asociada a los modelos IA, sus modos de uso y su composición dentro de pipelines, junto con la gestión de los archivos temporales generados durante las ejecuciones. Esta representación resulta especialmente útil para comprender el significado semántico de cada relación antes de su transformación al esquema relacional definitivo.

Diagrama del modelo relacional

La Figura C:2 muestra la transformación del modelo conceptual al modelo relacional. En este nivel, las entidades y relaciones anteriores pasan

a expresarse mediante tablas con sus claves primarias, claves foráneas y restricciones principales.

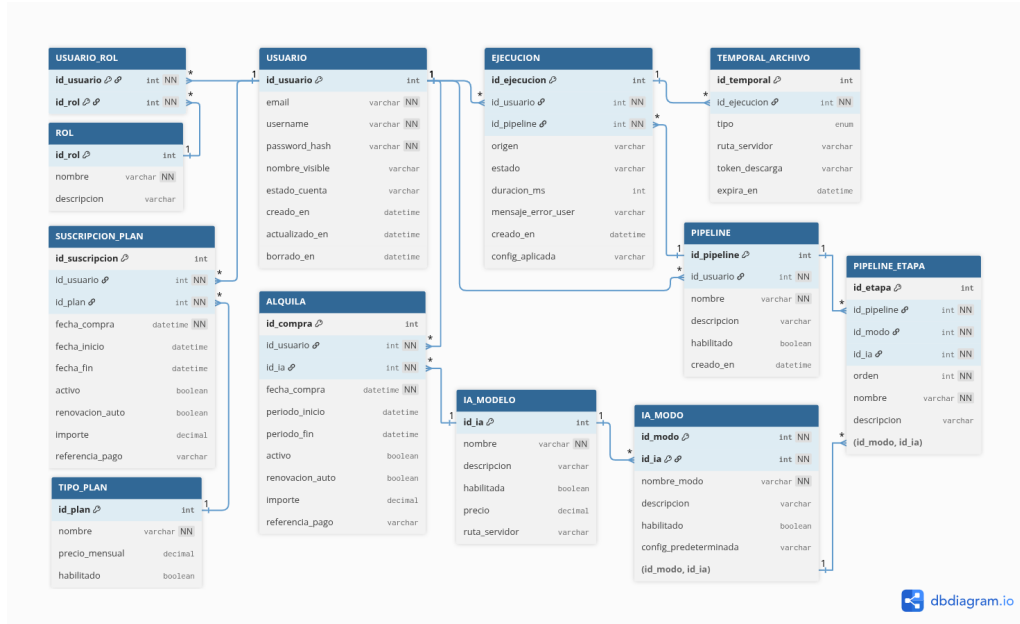


Figura C.2: Diagrama del modelo relacional del sistema

En este diagrama puede apreciarse con mayor precisión cómo se implementan las relaciones entre las distintas tablas. Por ejemplo, las relaciones de muchos a muchos se materializan mediante tablas intermedias como **USUARIO_ROL**, mientras que otras asociaciones se resuelven a través de claves foráneas directas, como sucede en **SUSCRIPCION_PLAN**, **ALQUILA**, **EJECUCION** o **PIPELINE_ETAPA**. Además, esta representación permite comprobar de forma más clara la estructura final sobre la que se aplican las restricciones de unicidad, integridad referencial y consistencia que se justifican a continuación.

A continuación se justifica cada una de las tablas del modelo..

Tabla USUARIO

La tabla **USUARIO** representa la cuenta de cada persona registrada en el sistema. Se trata de una tabla central, ya que gran parte del resto de entidades se relacionan directa o indirectamente con ella. Sus atributos se muestran en la Tabla C.1.

Atributo	Descripción
<code>id_usuario</code>	Identificador interno del usuario. Se define como clave primaria autoincremental para disponer de una referencia estable e independiente de los datos funcionales.
<code>email</code>	Correo electrónico del usuario. Es NOT NULL porque constituye un dato esencial para identificar la cuenta y se declara UNIQUE para impedir duplicidades.
<code>username</code>	Nombre de usuario del sistema. También es NOT NULL y UNIQUE porque debe existir siempre y no puede repetirse sin generar ambigüedad.
<code>password_hash</code>	Contraseña almacenada en formato hash. Es obligatoria porque forma parte de la autenticación y no se guarda en texto plano por seguridad.
<code>nombre_visible</code>	Nombre mostrado en la interfaz. Se permite NULL porque no es imprescindible para el funcionamiento interno del sistema.
<code>estado_cuenta</code>	Estado lógico de la cuenta. Es obligatorio y tiene el valor por defecto activa para evitar insertar usuarios sin estado definido.
<code>creado_en</code>	Fecha de creación del registro. Se marca como obligatoria y con valor por defecto para asegurar trazabilidad desde el alta.
<code>actualizado_en</code>	Fecha de última modificación. Se actualiza automáticamente para facilitar auditoría y seguimiento de cambios.
<code>borrado_en</code>	Fecha de borrado lógico. Se permite NULL porque solo debe informarse cuando el usuario deja de estar operativo sin eliminarse físicamente.

Tabla C.1: Atributos de la tabla USUARIO

Atributo	Descripción
<code>id_rol</code>	Identificador interno del rol. Se utiliza una clave primaria autoincremental para desacoplar la identificación interna del nombre funcional.
<code>nombre</code>	Nombre del rol. Es NOT NULL y UNIQUE para que cada rol quede claramente diferenciado y no existan duplicados lógicos.
<code>descripcion</code>	Texto descriptivo del rol. Se permite NULL porque es útil para documentar, pero no imprescindible para aplicar permisos.

Tabla C.2: Atributos de la tabla ROL

La clave primaria sobre `id_usuario` garantiza una identificación interna única y estable. Las restricciones **UNIQUE** sobre `email` y `username` se introducen porque ambos campos tienen significado funcional dentro del sistema: el primero evita que dos cuentas compartan el mismo correo electrónico y el segundo impide colisiones en el identificador visible del usuario. Por su parte, los campos obligatorios se han limitado a los estrictamente necesarios para operar, mientras que otros como `nombre_visible` o `borrado_en` se mantienen opcionales para no forzar información que no siempre debe existir.

Tabla ROL

La tabla **ROL** almacena los distintos perfiles de acceso del sistema, como puede ser usuario administrador, cliente o futuros roles como soporte. Sus atributos se muestran en la Tabla C.2.

La separación de **ROL** respecto de **USUARIO** permite gestionar los perfiles de forma flexible y evita codificar los roles como un simple valor fijo dentro de la cuenta. La unicidad del nombre se justifica porque no tendría sentido disponer de dos roles con la misma denominación, ya que eso introduciría ambigüedad tanto en la lógica del sistema como en la administración.

Tabla USUARIO__ROL

La tabla **USUARIO__ROL** materializa la asignación de roles a usuarios. Sus atributos se resumen en la Tabla C.3. Se trata de una tabla intermedia necesaria para representar una relación de muchos a muchos entre **USUARIO** y **ROL**.

Atributo	Descripción
<code>id_usuario</code>	Usuario al que se asigna el rol. Es obligatorio porque la asociación no tiene sentido sin un usuario existente.
<code>id_rol</code>	Rol asignado al usuario. También es obligatorio porque la asociación debe apuntar siempre a un rol concreto.

Tabla C.3: Atributos de la tabla `USUARIO_ROL`

La clave primaria compuesta (`id_usuario`, `id_rol`) evita que el mismo rol se asigne varias veces al mismo usuario. Esta solución es preferible a crear un identificador artificial adicional, ya que la propia combinación de ambos campos ya identifica unívocamente la asociación. En cuanto a las acciones de borrado, se utiliza `ON DELETE CASCADE` hacia `USUARIO` porque, si un usuario desaparece, sus asignaciones dejan de tener sentido; en cambio, hacia `ROL` se usa `ON DELETE RESTRICT` para impedir eliminar un rol que todavía está siendo utilizado.

Tabla `TIPO_PLAN`

La tabla `TIPO_PLAN` recoge el catálogo de planes disponibles en el sistema los cuales actualmente son pro y normal teniendo pro acceso a todos los modelos de IA y normal acceso a los que alquile, además de futuros planes que se implementen. Sus atributos se presentan en la Tabla C.4. Se separa de `SUSCRIPCION_PLAN` para distinguir entre el tipo abstracto de plan y la contratación concreta realizada por un usuario en un periodo determinado.

La restricción `UNIQUE` sobre `nombre` se justifica porque el catálogo no debe contener dos planes con la misma denominación. El `CHECK` sobre `precio_mensual` impide valores negativos, protegiendo la coherencia del dato económico desde la propia base de datos. El campo `habilitado` permite desactivar planes sin necesidad de eliminarlos, lo que conserva referencias históricas y simplifica la administración.

Tabla `SUSCRIPCION_PLAN`

La tabla `SUSCRIPCION_PLAN` registra cada suscripción efectiva de un usuario a un plan concreto. Sus atributos se recogen en la Tabla C.5. Se diferencia de `TIPO_PLAN` porque aquí no se define el plan, sino cada contratación realizada.

Atributo	Descripción
id_plan	Identificador interno del plan. Se utiliza como clave primaria autoincremental.
nombre	Nombre del plan. Es NOT NULL y UNIQUE porque cada elemento del catálogo debe quedar claramente identificado.
precio_mensual	Precio base mensual del plan. Se permite NULL para no obligar a informar un importe en planes gratuitos o todavía no definidos.
habilitado	Indica si el plan está disponible para nuevas contrataciones. Es obligatorio y tiene valor por defecto TRUE para evitar estados indefinidos.

Tabla C.4: Atributos de la tabla TIPO_PLAN

Las claves foráneas hacia **USUARIO** y **TIPO_PLAN** usan **RESTRICT** porque las suscripciones forman parte del histórico del sistema. El **CHECK** sobre fechas impide incoherencias temporales básicas y el **CHECK** sobre **importe** evita valores negativos.

Tabla IA__MODELO

La tabla **IA__MODELO** representa cada modelo de inteligencia artificial disponible en la aplicación. Sus atributos se muestran en la Tabla C.6. Esta tabla es una de las entidades principales del dominio, ya que el sistema se centra en permitir el acceso y la ejecución de distintos modelos IA.

La unicidad del nombre se establece para que cada modelo tenga una identidad lógica inequívoca. El **CHECK** sobre **precio** evita importes negativos. Además, el campo **habilitada** permite activar y desactivar modelos sin eliminarlos físicamente, lo que es útil en mantenimiento, pruebas o retirada temporal del servicio.

Tabla IA__MOD0

La tabla **IA__MOD0** recoge los distintos modos de funcionamiento de cada modelo IA. Sus atributos aparecen en la Tabla C.7. Esta separación permite reutilizar un mismo modelo con varias configuraciones o finalidades sin necesidad de duplicarlo en la tabla principal.

Atributo	Descripción
<code>id_suscripcion</code>	Identificador interno de la suscripción. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario suscrito. Es NOT NULL porque la suscripción siempre debe pertenecer a una cuenta concreta.
<code>id_plan</code>	Plan contratado. También es NOT NULL porque toda suscripción debe asociarse a un plan del catálogo.
<code>fecha_inicio</code>	Fecha de inicio de la suscripción. Se declara obligatoria con valor por defecto para registrar automáticamente el momento de alta.
<code>fecha_fin</code>	Fecha de finalización, cuando exista. Se permite NULL porque una suscripción puede estar activa o no tener todavía cierre registrado.
<code>activo</code>	Indica si la suscripción está vigente. Es obligatorio y tiene valor por defecto TRUE.
<code>renovacion_auto</code>	Indica si la renovación es automática. Es obligatorio y se inicia en FALSE por defecto.
<code>importe</code>	Importe cobrado en esa contratación concreta. Se permite NULL porque puede haber promociones o ajustes que no siempre se registren inicialmente.
<code>referencia_pago</code>	Referencia externa del pago asociado. Se deja opcional porque no tiene por qué existir siempre o puede no haberse informado todavía.

Tabla C.5: Atributos de la tabla SUSCRIPCION_PLAN

La restricción **UNIQUE** (`id_ia`, `nombre_modo`) evita que un mismo modelo tenga dos modos con el mismo nombre, lo que generaría ambigüedad. Además, se mantiene **UNIQUE** (`id_modo`, `id_ia`) porque esa combinación se usa posteriormente como referencia compuesta desde **PIPELINE_ETAPA**. La clave foránea hacia **IA_MODELO** utiliza **RESTRICT** para impedir eliminar un modelo si todavía conserva modos asociados.

Tabla ALQUILA

La tabla **ALQUILA** registra la adquisición individual de acceso a un modelo **IA** por parte de un usuario. Sus atributos se detallan en la Tabla C.8. Esta tabla resulta necesaria porque el sistema no solo contempla acceso

Atributo	Descripción
id_ia	Identificador interno del modelo. Se define como clave primaria autoincremental.
nombre	Nombre del modelo. Es NOT NULL y UNIQUE para evitar duplicidades semánticas.
descripcion	Descripción funcional del modelo. Se permite NULL porque no siempre es necesaria para la lógica interna.
habilitada	Indica si el modelo puede utilizarse. Es obligatorio y tiene valor por defecto TRUE para mantener un estado definido.
precio	Coste individual del acceso al modelo, si procede. Se permite NULL porque algunos modelos pueden no tener precio independiente.
ruta_servidor	Ruta o referencia interna utilizada por el backend para localizar el modelo. Se deja opcional para no forzar que todos los modelos se resuelvan exactamente igual.

Tabla C.6: Atributos de la tabla IA_MODELO

Atributo	Descripción
id_modo	Identificador interno del modo. Se define como clave primaria autoincremental.
id_ia	Modelo al que pertenece el modo. Es NOT NULL porque todo modo debe estar asociado a un modelo existente.
nombre_modo	Nombre del modo dentro del modelo. Es obligatorio porque identifica funcionalmente la variante de uso.
descripcion	Explicación opcional del modo. Se permite NULL porque no siempre será necesaria.
habilitado	Indica si el modo está disponible para su uso. Es obligatorio y tiene valor por defecto TRUE.
config_predeterminada	Configuración base del modo. Se almacena como TEXT y se permite NULL para mantener flexibilidad y no imponer una estructura fija.

Tabla C.7: Atributos de la tabla IA_MODO

Atributo	Descripción
<code>id_compra</code>	Identificador interno de la operación. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario que realiza la compra. Es NOT NULL porque toda operación debe tener titular.
<code>id_ia</code>	Modelo IA adquirido. También es NOT NULL porque la compra debe referirse a un recurso concreto.
<code>fecha_compra</code>	Fecha de la operación. Se declara obligatoria con valor por defecto para registrar automáticamente el momento del alta.
<code>periodo_inicio</code>	Fecha de inicio de la vigencia del acceso. Se permite NULL porque no todas las operaciones tienen por qué estar acotadas temporalmente del mismo modo.
<code>periodo_fin</code>	Fecha de finalización de la vigencia. También se permite NULL para mantener flexibilidad.
<code>activo</code>	Indica si el acceso está actualmente operativo. Es obligatorio y tiene valor por defecto TRUE.
<code>renovacion_auto</code>	Indica si la renovación del acceso es automática. Es obligatorio y se inicia por defecto en FALSE para no asumir renovaciones sin indicación expresa.
<code>importe</code>	Importe cobrado en la operación concreta. Es necesario para si cambia el precio de alquiler de modelo quede registrado cuando se pagó en el momento de la compra. Se permite NULL porque puede haber casos gratuitos, promocionales o pendientes de registrar.
<code>referencia_pago</code>	Referencia externa del pago. Se deja opcional porque no siempre existirá una referencia única o puede no haberse informado todavía.

Tabla C.8: Atributos de la tabla ALQUILA

por suscripción general, sino también compras o activaciones concretas de modelos.

Las claves foráneas hacia `USUARIO` e `IA_MODELO` emplean `RESTRICT` porque estas operaciones forman parte del histórico de negocio y no conviene perderlas automáticamente al eliminar entidades relacionadas. El `CHECK` sobre `importe` evita cantidades negativas y el `CHECK` sobre el periodo obliga a que, si ambas fechas existen, la fecha final no sea anterior a la inicial.

Atributo	Descripción
<code>id_pipeline</code>	Identificador interno del pipeline. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario propietario del pipeline. Se permite NULL porque el diseño contempla conservar el pipeline aunque desaparezca su propietario original.
<code>nombre</code>	Nombre del pipeline. Es NOT NULL porque resulta necesario para identificarlo funcionalmente.
<code>descripcion</code>	Descripción del flujo. Se permite NULL porque no siempre es imprescindible documentarlo.
<code>habilitado</code>	Indica si el pipeline puede utilizarse. Es obligatorio y tiene valor por defecto TRUE.
<code>creado_en</code>	Fecha de creación del pipeline. Se registra automáticamente para mantener trazabilidad temporal.

Tabla C.9: Atributos de la tabla PIPELINE

Tabla PIPELINE

La tabla PIPELINE representa un flujo de procesamiento configurable. Sus atributos se presentan en la Tabla C.9. En el diseño final destaca especialmente que el campo `id_usuario` puede ser nulo y que, si el usuario se elimina, su valor pasa a NULL en lugar de desaparecer el pipeline esto es debido a que si se crea un pipeline por un administrador y se elimina dicho usuario no tiene sentido que dejen de existir sus pipelines creados .

La restricción UNIQUE (`id_usuario`, `nombre`) evita que un mismo usuario tenga dos pipelines con el mismo nombre, lo que simplifica la gestión y evita ambigüedades. El hecho de permitir NULL en `id_usuario` y usar ON DELETE SET NULL responde a una decisión de conservación: si el usuario desaparece, el pipeline puede seguir existiendo como estructura lógica. No se utiliza CASCADE porque eso implicaría perder pipelines potencialmente reutilizables o relevantes para el histórico del sistema.

Tabla EJECUCION

La tabla EJECUCION registra cada ejecución concreta realizada en el sistema. Sus atributos se muestran en la Tabla C.10. Esta tabla es clave para la trazabilidad operativa, ya que permite saber quién ejecutó qué, cuándo lo hizo y con qué resultado.

Atributo	Descripción
<code>id_ejecucion</code>	Identificador interno de la ejecución. Se define como clave primaria autoincremental.
<code>id_usuario</code>	Usuario asociado a la ejecución. Es obligatorio para mantener responsabilidad y trazabilidad.
<code>id_pipeline</code>	Pipeline ejecutado. Es NOT NULL porque cada ejecución debe quedar vinculada a un flujo concreto.
<code>origen</code>	Punto de entrada desde el que se lanza la ejecución. Se permite NULL porque no siempre es necesario registrar esta información.
<code>estado</code>	Estado de la ejecución. Es obligatorio y su valor por defecto es <code>pendiente</code> , que representa el estado inicial natural.
<code>duracion_ms</code>	Duración de la ejecución en milisegundos. Se permite NULL porque puede no conocerse todavía al inicio del proceso.
<code>mensaje_error_user</code>	Mensaje de error orientado al usuario. Es opcional porque solo se utiliza cuando la ejecución falla.
<code>creado_en</code>	Fecha de creación de la ejecución. Se registra automáticamente para mantener trazabilidad temporal.
<code>config_aplicada</code>	Configuración real utilizada en la ejecución. Se permite NULL porque no siempre será necesario persistirla.

Tabla C.10: Atributos de la tabla EJECUCION

La clave foránea hacia `USUARIO` utiliza `RESTRICT` para no perder histórico operativo al eliminar usuarios. La clave foránea hacia `PIPELINE` también usa `RESTRICT`, lo que implica que no se puede eliminar un pipeline si existen ejecuciones asociadas a él. Esta decisión se justifica porque una ejecución debe seguir apuntando al pipeline concreto con el que fue lanzada y no conviene dejar esa referencia incompleta. El `CHECK` sobre `duracion_ms` evita registrar tiempos negativos.

Tabla `TEMPORAL_ARCHIVO`

La tabla `TEMPORAL_ARCHIVO` gestiona los archivos temporales asociados a una ejecución. Sus atributos aparecen en la Tabla C.11. Estos recursos pueden

Atributo	Descripción
<code>id_temporal</code>	Identificador interno del archivo temporal. Se define como clave primaria autoincremental.
<code>id_ejecucion</code>	Ejecución a la que pertenece el archivo. Es obligatorio porque el temporal no tiene sentido de forma independiente.
<code>tipo</code>	Tipo de archivo temporal. Se permite NULL porque puede haber casos en los que no se clasifique expresamente.
<code>ruta_servidor</code>	Ruta interna del archivo en el servidor. Se deja opcional para no imponer una única forma de gestión.
<code>token_descarga</code>	Token utilizado para acceso o descarga controlada. Se permite NULL porque no todos los archivos temporales tienen por qué exponerse.
<code>expira_en</code>	Fecha de expiración del recurso temporal. Es opcional porque puede calcularse o no utilizarse en todos los casos.

Tabla C.11: Atributos de la tabla TEMPORAL_ARCHIVO

representar archivos de entrada, de salida o intermedios y su naturaleza es claramente dependiente de la ejecución que los genera.

La restricción `UNIQUE` sobre `token_descarga` garantiza que, si existe, cada token identifique un único recurso. La clave foránea hacia `EJECUCION` emplea `ON DELETE CASCADE` porque aquí sí existe una dependencia total: si desaparece la ejecución, carece de sentido conservar sus archivos temporales. Es precisamente un caso claro y justificado de borrado en cascada.

Tabla PIPELINE_ETAPA

La tabla `PIPELINE_ETAPA` descompone cada pipeline en etapas ordenadas. Sus atributos se muestran en la Tabla C.12. Esta estructura permite representar un procesamiento secuencial y modular, en el que cada etapa invoca un modo concreto de un modelo IA.

La clave foránea hacia `PIPELINE` utiliza `ON DELETE CASCADE` porque las etapas dependen completamente del pipeline al que pertenecen. En cambio, la clave foránea compuesta (`id_modulo`, `id_ia`) hacia `IA_MODALIDAD` usa `RESTRICT` para impedir eliminar un modo que todavía está siendo utilizado. La restricción `UNIQUE` (`id_pipeline`, `nombre`, `orden`) evita duplicidades

Atributo	Descripción
<code>id_etapa</code>	Identificador interno de la etapa. Se define como clave primaria autoincremental.
<code>id_pipeline</code>	Pipeline al que pertenece la etapa. Es obligatorio porque una etapa no existe de forma autónoma.
<code>id_modo</code>	Modo IA utilizado en la etapa. Es NOT NULL porque la etapa debe invocar una operación concreta.
<code>id_ia</code>	Modelo IA asociado al modo referenciado. Es obligatorio para reforzar la coherencia de la referencia compuesta.
<code>orden</code>	Posición de la etapa dentro del pipeline. Es obligatorio porque el flujo debe tener un orden definido.
<code>nombre</code>	Nombre de la etapa. Es obligatorio para identificarla funcionalmente dentro del flujo.
<code>descripcion</code>	Descripción opcional de la finalidad de la etapa. Se permite NULL porque no siempre será necesaria.

Tabla C.12: Atributos de la tabla PIPELINE_ETAPA

problemáticas dentro de un mismo pipeline, y el **CHECK** (`orden >= 1`) obliga a que el orden sea positivo y coherente con la idea de secuencia.

Relaciones del modelo

Una vez descritas las tablas por separado, conviene explicar de forma conjunta las relaciones del modelo para entender mejor el sentido global del diseño.

- **USUARIO – USUARIO_ROL – ROL**: la asignación de roles se resuelve mediante una tabla intermedia porque la relación es de muchos a muchos. Un usuario puede tener varios roles y un rol puede estar asignado a varios usuarios.
- **USUARIO – SUSCRIPCION_PLAN**: un usuario puede contratar varias suscripciones a lo largo del tiempo, mientras que cada suscripción pertenece a un único usuario.
- **TIPO_PLAN – SUSCRIPCION_PLAN**: un tipo de plan puede estar asociado a muchas suscripciones, mientras que cada suscripción corresponde a un único plan.

- **USUARIO – ALQUILA:** un usuario puede realizar múltiples compras individuales de modelos IA, pero cada operación pertenece a un único usuario.
- **IA_MODELO – ALQUILA:** un modelo IA puede aparecer en muchas compras individuales, mientras que cada compra referencia a un único modelo.
- **IA_MODELO – IA_MODO:** un modelo puede disponer de varios modos, mientras que cada modo pertenece a un único modelo.
- **USUARIO – PIPELINE:** un usuario puede crear varios pipelines. Sin embargo, se permite que el pipeline sobreviva a la eliminación de su propietario mediante `ON DELETE SET NULL`. Esta decisión favorece la conservación de la estructura del flujo sin vincular su existencia estrictamente al usuario original.
- **PIPELINE – PIPELINE_ETAPA:** un pipeline puede contener varias etapas y estas dependen totalmente de él, por eso se utiliza borrado en cascada.
- **IA_MODO – PIPELINE_ETAPA:** cada etapa referencia un modo concreto mediante la pareja (`id_modo`, `id_ia`). Esta referencia compuesta garantiza que el modo utilizado pertenece realmente al modelo indicado.
- **USUARIO – EJECUCION:** un usuario puede generar muchas ejecuciones, mientras que cada ejecución pertenece a un único usuario.
- **PIPELINE – EJECUCION:** un pipeline puede ejecutarse muchas veces. En el diseño final se utiliza `RESTRICT` porque se considera importante que cada ejecución mantenga la referencia completa al pipeline con el que fue lanzada.
- **EJECUCION – TEMPORAL_ARCHIVO:** una ejecución puede generar varios archivos temporales y cada archivo temporal pertenece a una única ejecución. Aquí sí se justifica claramente el borrado en cascada, ya que el temporal depende por completo de la ejecución.

Valoración global del modelo

En conjunto, el modelo de datos final busca equilibrio entre flexibilidad, integridad y conservación del histórico. Las restricciones `UNIQUE` evitan duplicidades funcionales, los `CHECK` impiden valores inválidos sencillos y las

claves foráneas mantienen la coherencia entre entidades. Además, la elección entre **CASCADE**, **RESTRICT** y **SET NULL** no se ha hecho de manera uniforme, sino en función del significado de cada relación dentro del sistema.

De este modo, el modelo no solo representa correctamente las entidades del dominio, sino que también refleja decisiones de diseño orientadas a la robustez, la trazabilidad y la mantenibilidad de la aplicación.

C.3. Diseño arquitectónico

C.4. Diseño procedimental

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía
